Università di Napoli "Federico II"

Fluidodinamica Numerica

Ingegneria Aerospaziale

Simulation of 2D Viscous Flows using the $\psi - \zeta$ Model: Comparative Analysis of Numerical Schemes, Optimizations, and Streakline Tracking

Author:
Luigi Fragliasso

Supervisor:
Prof. G. Coppola

Academic Year 2025-2026

Contents

Abstract

This work discusses the numerical simulation of a two-dimensional incompressible viscous flow around an obstacle. The governing Navier-Stokes equations are solved using the $\psi - \zeta$ (streamfunction-vorticity) formulation on a collocated grid and spatially discretized via Finite Differences. Regarding the numerical solution, various implementation approaches are explored and compared: time integration is addressed using both single-step (Runge-Kutta) and multi-step (Adams-Bashforth) methods, while the efficiency of iterative solvers (SOR) is analyzed for the solution of the elliptic equation, including optimizations in C++. The main objective of this work is the implementation of a Lagrangian procedure for wake visualization through the computation of streaklines. These are generated by releasing tracer particles at successive time instants from a single injection point $\mathbf{x}_0$; by connecting the current positions (termed *isochronous positions*) of all previously released particles, the instantaneous representation of the streakline is obtained.

1 Governing Equations and Model Description

In this chapter, the fundamental equations governing fluid motion are described. We begin with the most general formulation of the Navier-Stokes equations (conservation of mass, momentum, and energy) and then introduce the simplifying assumptions of incompressibility, which lead to the decoupling of the energy equation. Finally, the system is specialized for the two-dimensional (2D) case using the $\psi - \zeta$ model.

1.1 The Navier-Stokes Equations

The fundamental equations describing the motion of a Newtonian viscous fluid under the influence of pressure, viscous forces, and external forces are the **Navier-Stokes equations**. In an Eulerian reference frame, denoting the velocity vector by $\mathbf{V}(\mathbf{x}, t)$, the conservation equations in vector form are:

Conservation of Mass (Continuity):

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \tag{1}$$

Conservation of Momentum:

$$\frac{\partial (\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V}\mathbf{V}) = \nabla \cdot \boldsymbol{\tau} + \rho \mathbf{f} \tag{2}$$

where:

- ρ is the fluid density,
- $\mathbf{f}$ represents the external body forces,
- $\boldsymbol{\tau}$ is the stress tensor.

The stress tensor $\boldsymbol{\tau}$ for a Newtonian fluid decomposes into an isotropic part (pressure) and a deviatoric part (viscous):

$$\boldsymbol{\tau} = -p\mathbf{I} + \boldsymbol{\tau}_d \tag{3}$$

where $\boldsymbol{\tau}_d$, assuming Stokes' hypothesis, is related to the strain rate by the dynamic viscosity μ:

$$\boldsymbol{\tau}_d = 2\mu \left[\frac{1}{2} \left(\nabla \mathbf{V} + (\nabla \mathbf{V})^T \right) - \frac{1}{3} (\nabla \cdot \mathbf{V}) \mathbf{I} \right] \tag{4}$$

Conservation of Total Energy:

$$\frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{u}) = \nabla \cdot (\boldsymbol{\tau} \cdot \mathbf{u}) - \nabla \cdot \mathbf{J}_q + \rho \mathbf{f} \cdot \mathbf{u} \tag{5}$$

where:

- $E = e + \frac{1}{2}|\mathbf{V}|^2$ is the total energy per unit volume, given by the sum of internal energy e and kinetic energy.
- $\mathbf{J}_q = -k\nabla T$ is the heat flux described by Fourier's Law, with k being the thermal conductivity and T the temperature.

1.2 Incompressibility Hypothesis

The **incompressible** fluid hypothesis implies that the density ρ is constant in time and space. Consequently, the continuity equation reduces to a kinematic constraint on the velocity field, which must be divergence-free ($\nabla \cdot \mathbf{V} = 0$). Adopting this hypothesis entails a fundamental change in the nature of the equations: the thermodynamic equation of state (linking pressure, density, and temperature) is no longer applicable. Otherwise, pressure would be uniquely determined by temperature, and there would not be sufficient degrees of freedom to satisfy the incompressibility constraint. In the incompressible model, pressure loses its thermodynamic significance and becomes a purely mechanical variable (a Lagrange multiplier) that adjusts instantaneously to ensure the flow field remains solenoidal. This leads to the **decoupling** of the energy equation from the dynamic system. Temperature T becomes a passive scalar, and the system to be solved reduces to only the mass and momentum equations:

$$\begin{cases} \nabla \cdot \mathbf{V} = 0 \\ \dfrac{\partial \mathbf{V}}{\partial t} + (\mathbf{V} \cdot \nabla)\mathbf{V} = -\dfrac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{V} + \mathbf{f} \end{cases} \tag{6}$$

1.3 $\psi - \zeta$ Model

Solving system (6) in primitive variables $(\mathbf{V}, p)$ presents numerical difficulties, mainly linked to the absence of an explicit evolutionary equation for pressure. However, restricting the study to **two-dimensional** flows, it is

possible to reformulate the problem more efficiently by introducing two new scalar variables: **Vorticity** (ζ) and the **Streamfunction** (ψ).
The introduction of the $\psi - \zeta$ model is motivated by decisive analytical and numerical simplifications:

1. **Elimination of pressure**: Through the use of vorticity, pressure disappears from the governing equations, removing the need to solve the velocity-pressure coupling.

2. **Automatic satisfaction of continuity**: The definition of the streamfunction identically guarantees that the condition $\nabla \cdot \mathbf{V} = \mathbf{0}$ is respected, without having to solve a dedicated differential equation.

3. **Reduction of unknowns**: The vector problem is transformed into a system of two coupled scalar equations.

1.3.1 Vorticity (ζ)

Vorticity is vectorially defined as the curl of the velocity field:

$$\boldsymbol{\omega} = \nabla \times \mathbf{u} \tag{7}$$

This definition allows the convective term of the momentum equation to be rewritten using the vector identity:

$$\mathbf{u} \cdot \nabla \mathbf{u} = \boldsymbol{\omega} \times \mathbf{u} + \nabla(|\mathbf{u}|^2/2). \tag{8}$$

Applying the curl operator ($\nabla\times$) to the entire momentum equation, the pressure gradient term vanishes (since $\nabla \times \nabla p = 0$). This yields the vorticity transport equation:

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + \mathbf{u} \cdot \nabla \boldsymbol{\omega} - \boldsymbol{\omega} \cdot \nabla \mathbf{u} = \nu \nabla^2 \boldsymbol{\omega} \tag{9}$$

In the specific case of a **two-dimensional** flow (on the xy plane), the vorticity vector is always perpendicular to the plane of motion. Defining the scalar component ω_z, the equation simplifies considerably as the **stretching** term ($\boldsymbol{\omega} \cdot \nabla \mathbf{u}$) vanishes. Defining the variable:

$$\zeta = -\omega_{\mathbf{z}} \tag{10}$$

opposite to the physical vorticity by convention of the model, the scalar transport equation becomes:

$$\frac{\partial \zeta}{\partial t} + \mathbf{u} \cdot \nabla \zeta = \nu \nabla^2 \zeta \tag{11}$$

1.3.2 Streamfunction (ψ)

Since the flow is incompressible, the velocity field is divergence-free ($\nabla \cdot \mathbf{u} = 0$). This implies the existence of a vector potential $\boldsymbol{\psi}$ such that $\mathbf{u} = \nabla \times \boldsymbol{\psi}$. In two dimensions, this potential has only the component normal to the plane, called the scalar **streamfunction** ψ. The velocity components are related to ψ by the relations:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \tag{12}$$

This definition identically satisfies the continuity equation. Substituting relations (12) into the definition of vorticity ($\omega_z = \partial v/\partial x - \partial u/\partial y$), a Poisson elliptic relation is obtained that links the kinematics of the field:

$$\nabla^{\mathbf{2}} \psi = -\omega_{\mathbf{z}} = \zeta \tag{13}$$

1.3.3 Final Dimensionless System

In order to generalize the simulation results and make them independent of the specific physical scale of the problem, it is necessary to perform a non-dimensionalization of the governing equations. Dimensionless quantities (indicated with an asterisk) are introduced by normalizing physical variables with respect to a characteristic length L_{ref} and a reference velocity u_{ref}:

$$\mathbf{x}^* = \frac{\mathbf{x}}{L_{ref}}, \quad \mathbf{u}^* = \frac{\mathbf{u}}{u_{ref}}, \quad t^* = \frac{t u_{ref}}{L_{ref}}, \quad \zeta^* = \frac{\zeta L_{ref}}{u_{ref}}, \quad \psi^* = \frac{\psi}{L_{ref} u_{ref}}$$

Substituting these definitions into the transport equation, the only dimensionless physical parameter governing the dynamics of viscous flow emerges, the **Reynolds Number**:

$$Re = \frac{u_{ref} L_{ref}}{\nu} \tag{14}$$

The final $\psi - \zeta$ system in dimensionless form (omitting the asterisks from now on for simplicity of notation) which will be solved numerically consists of the following two coupled equations:

$$\begin{cases} \dfrac{\partial \zeta}{\partial t} + \mathbf{u} \cdot \nabla \zeta = \dfrac{1}{Re} \nabla^2 \zeta & \text{(Transport Eq.)} \\ \nabla^2 \psi = \zeta & \text{(Poisson Eq.)} \end{cases} \tag{15}$$

2 Flow Lines

In addition to determining scalar and vector fields, a crucial aspect of computational fluid dynamics is flow visualization. To understand flow topology, especially in unsteady (time-dependent) regimes, it is fundamental to distinguish between different characteristic curves: streamlines, pathlines, and streaklines. Although in a steady flow these three curves coincide, in the case of unsteady flows they differ substantially and provide different physical information.

2.1 Streamlines

A streamline is defined as an instantaneous curve tangent at every point to the velocity vector $\mathbf{u}(\mathbf{x}, t)$ at a fixed instant t. It provides a "photographic" image of the Eulerian flow field frozen in time. Mathematically, denoting by $d\mathbf{l}$ the infinitesimal element oriented along the line, the tangency condition is expressed as a null cross product:

$$\mathbf{u} \times d\mathbf{l} = 0 \tag{16}$$

In the context of the two-dimensional $\psi - \zeta$ model, streamlines coincide with the contour lines (level curves) of the scalar function ψ. To demonstrate this, let us consider the gradient of ψ, which is a vector lying in the $x - y$ plane of motion:

$$\nabla\psi = \left(\frac{\partial\psi}{\partial x}, \frac{\partial\psi}{\partial y}\right) \tag{17}$$

By performing the dot product between the velocity vector $\mathbf{u} = (u, v)$ and the gradient of ψ, and substituting the definitions $u = \partial\psi/\partial y$ and $v = -\partial\psi/\partial x$, we obtain:

$$\mathbf{u} \cdot \nabla\psi = u\frac{\partial\psi}{\partial x} + v\frac{\partial\psi}{\partial y} = \left(\frac{\partial\psi}{\partial y}\right)\frac{\partial\psi}{\partial x} + \left(-\frac{\partial\psi}{\partial x}\right)\frac{\partial\psi}{\partial y} = 0 \tag{18}$$

The null result of the dot product implies that the velocity vector is everywhere **orthogonal** to the gradient of the streamfunction ($\mathbf{u} \perp \nabla\psi$). Since, by definition, the velocity $\mathbf{u}$ is tangent to the streamlines, it follows that the gradient $\nabla\psi$ (which represents the direction of maximum variation of the scalar) is normal to the streamlines. This means that the variation of ψ along the direction of motion is zero. Therefore, ψ remains constant along every streamline ($\psi = \text{const}$), which geometrically coincide with the contour lines (isolines) of the streamfunction.

2.2 Pathlines

A pathline is the locus of points visited by a single fluid particle during its motion. It is an intrinsically Lagrangian concept that describes the time history of a fluid element. Denoting the particle position by $\mathbf{x}_p(t)$, the pathline is determined by the time integration of the local velocity:

$$\frac{d\mathbf{x}_p}{dt} = \mathbf{u}(\mathbf{x}_p(t), t), \quad \text{with } \mathbf{x}_p(t_0) = \mathbf{x}_{start} \tag{19}$$

Unlike streamlines, pathlines do not depend only on the instantaneous field, but on the entire temporal evolution of the velocity field.

2.3 Streaklines

Emission lines, or *streaklines*, are the locus of positions occupied at a given instant t by all fluid particles that have passed through a specific fixed spatial point $\mathbf{x}_{in}$ (injection point) at different past instants. This definition corresponds exactly to what is observed experimentally by continuously injecting a dye (or smoke) into a point in the fluid: the visible line is formed by the set of dye particles released previously. From a mathematical and computational point of view, a streakline at instant t is constructed by joining the **isochronous positions** $\mathbf{x}_k(t)$ of a series of particles k, each released at time $t_k = t_0 + k\Delta t$. Each particle satisfies its own trajectory equation:

$$\begin{cases} \dfrac{d\mathbf{x}_k}{dt} = \mathbf{u}(\mathbf{x}_k(t), t) \\ \mathbf{x}_k(t_k) = \mathbf{x}_{in} \end{cases} \tag{20}$$

The visualization of streaklines is the primary objective of this work, as it allows for highlighting coherent structures and wake dynamics (such as von Kármán vortices) more effectively than simple instantaneous streamlines.

3 Numerical Methodology and Implementation

The solution of the governing system (15) is not obtainable analytically for arbitrary geometries; therefore, numerical analysis is employed. This section describes the spatial and temporal discretization techniques adopted to transform the partial differential equations into an algebraic system solvable by computer.

3.1 Spatial Discretization

The continuous physical domain is discretized using a structured and uniform Cartesian grid (*collocated* grid), with a constant spatial step $h = \Delta x = \Delta y$. The variables ψ and ζ are defined at the geometric nodes of the grid (i, j). Spatial derivatives are approximated using second-order centered **Finite Difference** (Finite Difference Method, FDM) schemes.

3.1.1 Discretization of the Diffusive Term

For the Laplace operator (∇^2), the classic "5-point stencil" is used:

$$\nabla^2 f|_{i,j} \approx \frac{f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4f_{i,j}}{h^2} \tag{21}$$

3.1.2 Discretization of the Convective Term

The discretization of the nonlinear convective term is a critical aspect for the stability and physical fidelity of the simulation. Analytically, this term can be expressed in two equivalent forms (thanks to incompressibility $\nabla \cdot \mathbf{u} = 0$):

- **Advective Form:** $\mathbf{u} \cdot \nabla\zeta$
- **Divergence Form:** $\nabla \cdot (\mathbf{u}\zeta)$

The use of one form or the other in the discrete domain depends on the conservation properties that the numerical scheme must respect. Using a centered finite difference scheme on a uniform grid, the discretizations take the following form:

1. **Advective Form:**

$$\mathbf{u} \cdot \nabla \zeta|_{i,j} \approx u_{i,j} \frac{\zeta_{i+1,j} - \zeta_{i-1,j}}{2h} + v_{i,j} \frac{\zeta_{i,j+1} - \zeta_{i,j-1}}{2h} \tag{22}$$

2. **Divergence Form:**

$$\nabla \cdot (\mathbf{u}\zeta)|_{i,j} \approx \frac{u_{i+1,j}\zeta_{i+1,j} - u_{i-1,j}\zeta_{i-1,j}}{2h} + \frac{v_{i,j+1}\zeta_{i,j+1} - v_{i,j-1}\zeta_{i,j-1}}{2h} \tag{23}$$

The objective is to ensure that the discretization does not introduce spurious quantities into the domain; that is, the conservation of physical invariants is required: the **linear invariant** (total vorticity ζ) and the **quadratic invariant** (enstrophy $\mathcal{E} \propto \zeta^2$). For this to occur, the global integral of the convective term over the domain Ω must vanish (assuming null or periodic boundary fluxes):

$$\int_\Omega \nabla \cdot (\mathbf{u}\zeta)\, d\Omega = 0 \quad \text{and} \quad \int_\Omega \mathbf{u} \cdot \nabla \zeta \, d\Omega = 0 \tag{24}$$

Analysis of the Linear Invariant: To evaluate the conservation of physical properties, it is useful to express the discrete convective terms in matrix notation. Let us introduce the diagonal velocity matrices $U = \text{diag}(u_{i,j})$ and $V = \text{diag}(v_{i,j})$, and the matrices D_x and D_y representing the discrete differentiation operators (central differences). The two forms of the convective term can thus be written as matrix operations on the vorticity vector $\boldsymbol{\zeta}$:

- **Divergence Form:** corresponds to the vector $(D_x U + D_y V)\boldsymbol{\zeta}$.
- **Advective Form:** corresponds to the vector $(U D_x + V D_y)\boldsymbol{\zeta}$.

It can be demonstrated that the *Divergence Form* inherently conserves the linear invariant (the telescoping sum of fluxes vanishes) without the need for further assumptions. The *Advective Form*, on the other hand, conserves it only under the hypothesis of using *central schemes* and if the discrete velocity field exactly satisfies the *incompressibility* condition.

Analysis of the Quadratic Invariant (Enstrophy): The conservation of enstrophy requires that the convective term does not contribute to the variation of the system's rotational energy. In algebraic terms, this implies that the associated quadratic forms must be zero. However, using the introduced notation, it is observed that:

- For the Divergence form, the associated quadratic form is $\boldsymbol{\zeta}^T(D_xU + D_yV)\boldsymbol{\zeta}$. This is not zero because the resulting matrix is not skew-symmetric.
- For the Advective form, the associated quadratic form is $\boldsymbol{\zeta}^T(UD_x + VD_y)\boldsymbol{\zeta}$. Similarly, this is generally not zero either.

Neither of the two fundamental forms, taken individually, is capable of conserving enstrophy, leading to potential nonlinear numerical instabilities over long times.

Solution: Skew-Symmetric Form It is proven that the two schemes present a discretization error regarding energy conservation that is equal in magnitude but opposite in sign. Therefore, by performing a linear combination of the two forms with a coefficient $\alpha = 0.5$:

$$\text{Conv}_{Skew} = \frac{1}{2}(\text{Advective}) + \frac{1}{2}(\text{Divergence}) \tag{25}$$

the **Skew-Symmetric** form is obtained. This formulation guarantees that the resulting discrete matrix is skew-symmetric, identically ensuring the conservation of the quadratic invariant (enstrophy) and guaranteeing the robust stability of the simulation.

3.2 Boundary Conditions

The elliptic-parabolic problem requires specific conditions on the boundaries of the computational domain.

Streamfunction (ψ): On solid walls (both the channel walls and the obstacle surface), the normal velocity must be zero. Since the difference in the value of ψ between two points represents the flow rate passing between them, solid walls are streamlines with a constant value ($\psi = \text{const}$).

- **Channel walls (South/North):** We impose $\psi_{sud} = 0$ as a reference value and $\psi_{nord} = Q$. Although the streamfunction is defined up to an arbitrary constant, the difference in value between two streamlines, as previously stated, represents the volumetric flow rate flowing between them. Consequently, it is necessary to impose a jump equal to Q between the walls; if we had imposed the same value on both sides (e.g.,

both zero), the resulting net flow rate would have been zero, preventing the development of a flow from left to right.

- **Obstacle:** On the surface of the obstacle, we impose $\psi_{ost} = Q/2$. This value is chosen for reasons of symmetry and flow continuity. If we had imposed $\psi_{ost} = 0$ (equal to the South wall), the entire region between the lower wall and the obstacle would have become a single streamline with a constant value; physically, this would have prevented fluid passage in that zone, forcing the entire flow to pass exclusively through the upper part of the channel, drastically altering the physics of the problem.

- **Inlet/Outlet (East/West):** We impose a linear profile $\psi(y) = u_{ref}y$. This choice generates a uniform velocity field at the inlet; indeed, recalling the definition $u = \partial\psi/\partial y$, the constant slope of the linear profile corresponds exactly to the mean velocity, calculated as the ratio between the flow rate difference at the boundaries and the domain height:

$$u_{ref} = \frac{\partial\psi}{\partial y} = \frac{\psi_{nord} - \psi_{sud}}{L_y} = \frac{Q - 0}{L_y} = \frac{Q}{L_y}$$

Vorticity (ζ): There is no direct physical boundary condition for vorticity. Therefore, a numerical relation derived from the Taylor series expansion of the streamfunction in the vicinity of the outer wall and the obstacle is employed. Considering a fluid node n adjacent to a wall node w, the second-order expansion is:

$$\psi_n = \psi_w + \left.\frac{\partial\psi}{\partial n}\right|_w h + \left.\frac{\partial^2\psi}{\partial n^2}\right|_w \frac{h^2}{2} + \mathcal{O}(h^3) \tag{26}$$

Recalling that the tangential velocity is $u_w = (\partial\psi/\partial n)_w$ and that, according to the Poisson equation at the wall, $\zeta_w = (\partial^2\psi/\partial n^2)_w$, the wall vorticity can be expressed as:

$$\zeta_w = \frac{2(\psi_n - \psi_w - u_w h)}{h^2} + \mathcal{O}(h) \tag{27}$$

Imposing the physical **no-slip** condition on solid walls, the tangential velocity vanishes ($u_w = 0$). This yields the famous **Thom's Formula**:

$$\zeta_w = \frac{2(\psi_n - \psi_w)}{h^2} \tag{28}$$

This formula allows vorticity to be physically generated on the body surface (due to velocity gradients) and subsequently diffused and convected into the wake.

3.3 Geometry Handling

The presence of obstacles immersed in the flow (cylinder or airfoil) within a rectangular Cartesian grid is managed via a topological matrix, denoted as $\mathbf{G}$. This matrix acts as a "mask" to distinguish the nature of each node (i, j):

- **Fluid Domain ($G > 0$):** Nodes where the transport and Poisson equations are actively solved.
- **Obstacle ($G = -1$):** Nodes internal to the solid body, excluded from the fluid calculation.
- **Boundaries ($G = 0$):** Boundary nodes where boundary conditions are imposed.

This technique allows for the treatment of even more complex geometries simply by modifying the initial definition of the matrix $\mathbf{G}$, without altering the equation solver.

3.4 Time Integration

Spatial discretization leads to a system of Ordinary Differential Equations (ODEs) for the nodal values of vorticity $\boldsymbol{\zeta}$. Defining the spatial operator $\mathcal{F}(\boldsymbol{\zeta})$ which encompasses the convective term (matrix $\mathbf{C}$) and the diffusive term (Laplacian matrix $\mathbf{L}$):

$$\mathcal{F}(\boldsymbol{\zeta}) = -\mathbf{C}(\boldsymbol{\zeta})\boldsymbol{\zeta} + \frac{1}{Re}\mathbf{L}\boldsymbol{\zeta} \tag{29}$$

the temporal evolution is governed by the equation $d\boldsymbol{\zeta}/dt = \mathcal{F}(\boldsymbol{\zeta})$. For time advancement, the explicit **fourth-order Runge-Kutta (RK4)** method is adopted. This scheme is defined by the following *Butcher Tableau*, which specifies the weights and time nodes for the four intermediate stages:

$$
\begin{array}{c|cccc}
0 & 0 & 0 & 0 & 0 \\
1/2 & 1/2 & 0 & 0 & 0 \\
1/2 & 0 & 1/2 & 0 & 0 \\
1 & 0 & 0 & 1 & 0 \\
\hline
 & 1/6 & 1/3 & 1/3 & 1/6
\end{array}
\tag{30}
$$

Given the known state at time t_n, denoted as $\boldsymbol{\zeta}_{old}$, the algorithm calculates the solution at the next step $\boldsymbol{\zeta}_{new}$ by evaluating four intermediate slopes (f_1, f_2, f_3, f_4) and predicted intermediate states $(\boldsymbol{\zeta}_1, \boldsymbol{\zeta}_2, \boldsymbol{\zeta}_3, \boldsymbol{\zeta}_4)$. The explicit procedure for a step Δt is as follows:

$$
\begin{aligned}
\text{Stage 1:} \quad & \boldsymbol{\zeta}_1 = \boldsymbol{\zeta}_{old} \\
& f_1 = \mathcal{F}(\boldsymbol{\zeta}_1)
\end{aligned}
\tag{31}
$$

$$
\begin{aligned}
\text{Stage 2:} \quad & \boldsymbol{\zeta}_2 = \boldsymbol{\zeta}_{old} + \frac{\Delta t}{2} f_1 \\
& f_2 = \mathcal{F}(\boldsymbol{\zeta}_2)
\end{aligned}
\tag{32}
$$

$$
\begin{aligned}
\text{Stage 3:} \quad & \boldsymbol{\zeta}_3 = \boldsymbol{\zeta}_{old} + \frac{\Delta t}{2} f_2 \\
& f_3 = \mathcal{F}(\boldsymbol{\zeta}_3)
\end{aligned}
\tag{33}
$$

$$
\begin{aligned}
\text{Stage 4:} \quad & \boldsymbol{\zeta}_4 = \boldsymbol{\zeta}_{old} + \Delta t f_3 \\
& f_4 = \mathcal{F}(\boldsymbol{\zeta}_4)
\end{aligned}
\tag{34}
$$

The updated solution at time t_{n+1} is finally obtained via a weighted average of the calculated slopes:

$$
\boldsymbol{\zeta}_{new} = \boldsymbol{\zeta}_{old} + \Delta t \left[\frac{1}{6}(f_1 + f_4) + \frac{1}{3}(f_2 + f_3) \right] \tag{35}
$$

3.4.1 Simulation Algorithm and Lagrangian Tracking

The same RK4 integration strategy is applied to evolve the position of the tracer particles $\mathbf{x}_p$ for the visualization of streaklines. The simulation algorithm proceeds iteratively according to the following steps:

1. **Boundary Vorticity Update:** Before each RK4 stage, the ζ values on the wall nodes are recalculated using Thom's formula.

2. **Eulerian Integration:** The new vorticity field $\boldsymbol{\zeta}_{new}$ is calculated by solving the transport equation according to the RK4 scheme described above.

3. **Elliptic Solution:** The kinematic field is updated by solving the sparse linear system associated with the Poisson equation:

$$\mathbf{L}\boldsymbol{\psi}_{new} = \boldsymbol{\zeta}_{new} \implies \boldsymbol{\psi}_{new} = \mathbf{L}^{-1}\boldsymbol{\zeta}_{new} \tag{36}$$

4. **Velocity Calculation:** The discrete velocity field $\mathbf{U}_{grid}$ on nodes (i, j) is derived by applying central differences to the updated streamfunction.

5. **Lagrangian Integration (Streaklines):** In parallel, the motion of passive particles is tracked by solving $d\mathbf{x}_p/dt = \mathbf{u}(\mathbf{x}_p)$. Since the particle position $\mathbf{x}_p$ varies continuously and almost never coincides with the grid nodes, the velocity $\mathbf{u}(\mathbf{x}_p)$ required to calculate the RK4 slopes f_i must be reconstructed locally.

   A **bilinear interpolation** is used, based on the four nodes of the cell (i, j) containing the particle. The idea behind this method is to approximate the velocity profile as linear along the two axial directions within each cell, ensuring field continuity (C^0) across element boundaries.

   Having defined the normalized local coordinates $\tilde{x} = (x_p - x_i)/h$ and $\tilde{y} = (y_p - y_j)/h$, which vary between 0 and 1, the interpolated velocity is expressed in matrix form as:

$$\mathbf{u}(\mathbf{x}_p) \approx \begin{bmatrix} 1-\tilde{x} & \tilde{x} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{u}_{i,j} & \mathbf{u}_{i,j+1} \\ \mathbf{u}_{i+1,j} & \mathbf{u}_{i+1,j+1} \end{bmatrix} \cdot \begin{bmatrix} 1-\tilde{y} \\ \tilde{y} \end{bmatrix} \tag{37}$$

   Physical interpretation of weights: The formula represents a weighted average of the velocities at the vertices. The coefficients act as geometric weights inversely proportional to the distance between the particle and the node:

   - The term $(1-\tilde{x})(1-\tilde{y})$ is the weight associated with node (i, j). If the particle is located exactly on node (i, j) (thus $\tilde{x} = 0, \tilde{y} = 0$),

this weight equals 1 and the interpolated velocity coincides exactly with the nodal one.

- As the particle moves away from a node, the corresponding weight decreases linearly, transferring "influence" to the nodes it is approaching.

This procedure ensures that the particle is most affected by the velocity field of the node closest to it, allowing for a smooth reconstruction of trajectories even on discrete grids.

This instantaneous velocity value feeds the Runge-Kutta stages for the update of the position $\mathbf{x}_p$.

4 Optimizations and Implementation Variants

In addition to the standard implementation described in the previous chapters, several numerical variants and software optimizations were explored and integrated into the code. These modifications aim to improve computational efficiency (reducing calculation times and memory usage) or to experiment with different integration strategies.

4.1 Poisson Solver: SOR Iterative Method

In the baseline implementation, the Poisson equation for the streamfunction ($\nabla^2\psi = \zeta$) is solved using a direct method, which involves the inversion (or factorization) of the large sparse Laplacian matrix $\mathbf{L}$. Although very efficient for moderately sized grids, this approach presents scalability limits:

- **Memory Footprint:** The matrix $\mathbf{L}$, despite being sparse, requires the allocation of a significant amount of RAM as the number of nodes $N = N_x \times N_y$ grows (non-negligible spatial complexity for indexing).
- **Computation Time:** Direct factorization becomes computationally expensive as the grid fineness increases.

To overcome such issues on very fine meshes, an alternative solver based on the **SOR (Successive Over-Relaxation)** iterative method was implemented. The algorithm updates the value of $\psi_{i,j}$ locally without ever assembling the global matrix:

$$\psi_{i,j}^{k+1} = (1-\omega)\psi_{i,j}^{k} + \omega \frac{\psi_{i+1,j}^{k} + \psi_{i-1,j}^{k+1} + \psi_{i,j+1}^{k} + \psi_{i,j-1}^{k+1} - h^2\zeta_{i,j}}{4} \tag{38}$$

where ω is the relaxation factor ($1 < \omega < 2$) chosen to accelerate convergence. Although performance gains are marginal for the grids used in this study, the iterative approach guarantees $O(N)$ (linear) memory usage, proving advantageous for high-resolution simulations (High-Performance Computing).

4.2 Hardware Acceleration: C++ Implementation (MEX)

Iterative methods like SOR require iterating through all grid nodes for thousands of iterations at each time step. In an interpreted language like MATLAB, executing nested `for` loops entails significant **overhead** (interpretation

cost) that drastically slows down the simulation. To mitigate this bottleneck, the computational kernel of the SOR solver (`PoissonSolver_SOR`) was rewritten in **C++** and interfaced with MATLAB via the **MEX (MATLAB Executable)** functionality. The advantages of this hybrid solution are:

1. **Compiled Code:** C++ is translated directly into machine language, eliminating interpreter overhead.
2. **Memory Management:** The use of pointers and direct access to contiguous memory (respecting MATLAB's *column-major* ordering) allows for optimal use of the CPU cache.
3. **In-Place Update:** The ψ matrix is updated directly in memory without intermediate copies.

The use of the MEX routine led to a reduction in solver computation time of over an order of magnitude compared to the counterpart written in pure MATLAB script.

4.3 Multi-Step Time Integration (Adams-Bashforth)

As an alternative to the *single-step* Runge-Kutta 4 (RK4) scheme, the implementation of explicit *multi-step* methods, such as **Adams-Bashforth** (e.g., AB2 or AB3), was evaluated. While RK4 requires 4 evaluations of the residual function $\mathcal{F}(\boldsymbol{\zeta})$ for each time step (resulting in high costs if $\mathcal{F}$ is complex), an Adams-Bashforth method uses the temporal "history" of the solution to advance:

$$\boldsymbol{\zeta}^{n+1} = \boldsymbol{\zeta}^n + \Delta t \sum_{j=0}^{k} b_j \mathcal{F}(\boldsymbol{\zeta}^{n-j}) \tag{39}$$

For example, the AB2 scheme requires only one function evaluation per step, reusing the one calculated at the previous step. This approach reduces the computational cost per single time step, at the price of a generally smaller stability domain and the need for a start-up procedure using a single-step method (like RK4) for the first few instants.

5 Results Analysis: Wake Evolution and Conservation

This section presents the temporal evolution of the flow field through the visualization of *streaklines*. Tracer particles are continuously released from a set of injectors located upstream of the obstacle. The coloring of the lines is not merely aesthetic but represents the intensity and sign of the local vorticity ζ:

- **Red:** Positive vorticity ($\zeta > 0$, counter-clockwise rotation);
- **Blue:** Negative vorticity ($\zeta < 0$, clockwise rotation);
- **Black/White:** Irrotational flow ($\zeta \approx 0$).

The superimposed black arrows represent the instantaneous velocity vector field.

5.1 Initial Phase and Transient ($t = 0.3$ - $t = 1.0$)

In the very first instants of the simulation, the flow maintains symmetry with respect to the horizontal axis passing through the center of the cylinder.

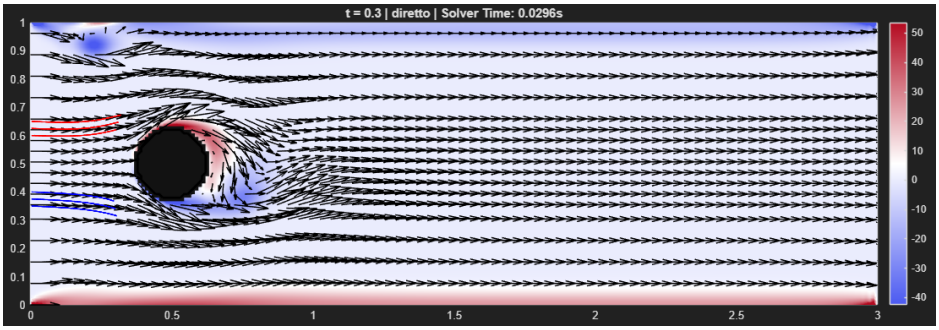


Figure 1: Time $t = 0.3s$: Formation of the symmetric recirculation zone.

As observed in Figure 1 ($t = 0.3s$), the streamlines adhere to the front of the cylinder and separate at the rear, forming two counter-rotating recirculation bubbles (twin vortices) that initially remain attached to the body. In this phase, the *streaklines* are still regular and exhibit no oscillations, indicating an unstable quasi-steady regime.

Subsequently (Figure 2, $t = 1s$), numerical perturbations (or physical ones, introduced deliberately) begin to amplify. The recirculation bubbles elongate, and the specular symmetry begins to break. The *streaklines* start to undulate in the far wake region, a prelude to the von Kármán instability.

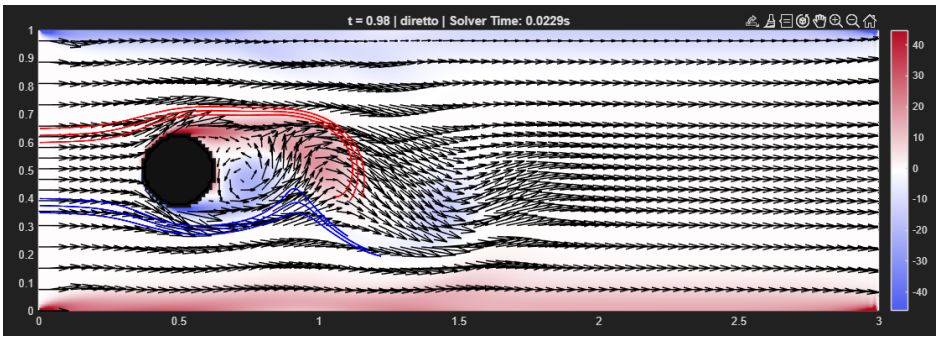


Figure 2: Time $t = 1s$: Breakdown of symmetry and onset of instability.

5.2 Fully Developed Regime ($t = 5.0s$)

At long times, the flow reaches a stable periodic regime characterized by alternating vortex shedding.

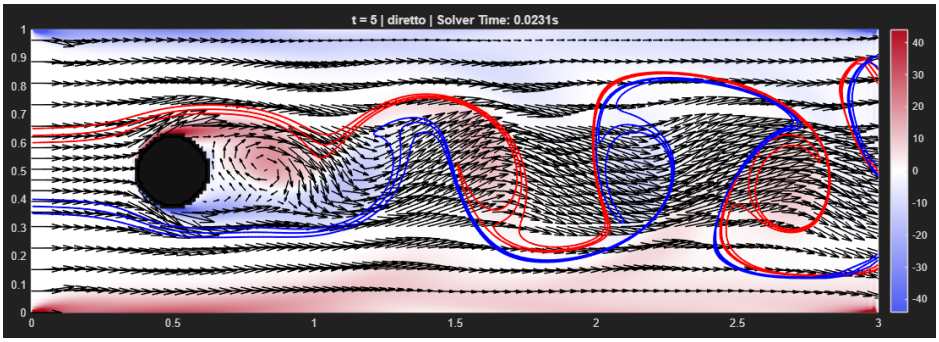


Figure 3: Time $t = 5.0$: Fully developed von Kármán wake.

Figure 3 clearly shows the **von Kármán Vortex Street**. The alternation of coherent vortical structures is evident: a clockwise vortex (blue) sheds from the upper part, followed by a counter-clockwise vortex (red) from the lower part. The *streaklines* are no longer taut lines but spiral inside the vortex cores, highlighting mass transport and strong mixing in the wake. This

behavior confirms the numerical model's capability to capture convective non-linearities.

5.3 Enstrophy Analysis and Initial Singularity

To evaluate the robustness and conservativeness of the numerical scheme, the evolution of the **Global Enstrophy** $\Omega(t)$ was monitored, defined (in 2D) as the integral of the squared vorticity over the domain:

$$\Omega(t) = \frac{1}{2} \int_D \zeta^2 \, dA \approx \frac{h^2}{2} \sum_{i,j} \zeta_{i,j}^2 \tag{40}$$

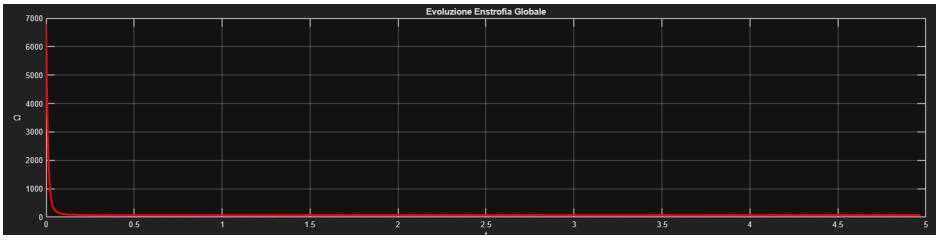


Figure 4: Temporal evolution of global enstrophy.

From the graph in Figure 4, a characteristic behavior is observed:

1. **Singularity at startup ($t \to 0$):** An extremely high peak in enstrophy is recorded in the very first instants. This phenomenon is both physical and numerical, due to the *Impulsive Start*. At time $t = 0$, the fluid is at rest (or has uniform velocity) while the walls instantaneously impose the no-slip condition ($u = 0$). Mathematically, this creates a velocity discontinuity on the body surface, corresponding to a boundary layer of zero thickness and thus a velocity gradient (and vorticity) tending to infinity ($\zeta \propto \partial u / \partial n \to \infty$).

2. **Viscous Decay:** Immediately after the start, the diffusive term of the Navier-Stokes equation ($\nu \nabla^2 \zeta$) acts by "smearing" the vorticity from the wall towards the interior of the domain. This regularizes the solution, thickens the boundary layer, and causes the enstrophy value to drop towards finite values.

3. **Asymptotic Regime:** For $t > 1$, the enstrophy stabilizes at a nearly constant average value, indicating an equilibrium between vorticity production at the walls and viscous dissipation within the domain.

6 Implementation and Case Study Analysis

In this chapter, several numerical approaches for solving the ψ–ζ system are analyzed and compared. The analysis is divided into two main sections: the first evaluates the efficiency of the solvers for the elliptic Poisson equation (direct vs. iterative methods), while the second examines the impact of different time integration schemes on the solution's stability and accuracy.

All case studies share:

- the same spatial discretization on a collocated grid;
- the same treatment of the convective term (skew-symmetric form);
- identical boundary conditions and physical parameters.

6.1 Simulation Setup

All simulations, unless otherwise indicated, refer to flow around a circular cylinder in a rectangular channel. The geometry and grid are defined by the parameters reported in Table 1.

Parameter	Symbol	Value
Domain (Height)	L_y	1.0
Domain (Width)	L_x	3.0 ($AR = 3$)
Cylinder Radius	R	$L_y/8$
Center Position	(x_c, y_c)	$(L_x/6, L_y/2)$
Nodes in y	N_y	80
Nodes in x	N_x	240
Spatial Step	h	$1/80 \approx 0.0125$
Reynolds Number	Re	500
Final Time	T	30
Convective CFL	Cou	0.3
Diffusive CFL	β	0.5

Table 1: Geometric, physical, and numerical parameters of the reference simulation.

The time step Δt is calculated dynamically at the beginning of the simulation to ensure stability according to the most restrictive criterion:

$$\Delta t \leq \min\left(\frac{Cou * th}{u_{max}}, \frac{\beta * Re * h^2}{4}\right) \tag{41}$$

6.2 Analysis of Solvers for the Poisson Equation

The resolution of the elliptic equation $\nabla^2\psi = \zeta$ represents the dominant computational cost in each time step. Two radically different approaches were compared: direct methods and iterative methods.

In order to perform an objective comparison between the different implemented solution strategies, an indicator related to computational efficiency was defined.

6.2.1 Efficiency: Average Computation Time per Step ($\bar{T}_{step}$)

Since the resolution of the Poisson equation ($\nabla^2\psi = \zeta$) represents the most burdensome operation of the entire algorithm (the "bottleneck"), the code was instrumented to measure the execution time of this specific phase at

each time iteration. Using system timing functions (`tic` and `toc`), the time interval T_i required by the solver to complete the calculation at the i-th step is recorded. The comparison parameter used is the **Average Time per Step**, calculated as the arithmetic mean over the N_t total steps of the simulation:

$$\bar{T}_{step} = \frac{1}{N_t} \sum_{i=1}^{N_t} T_i \tag{42}$$

- For the **Direct Method**, this time includes the resolution of the sparse linear system (back-substitution). Since the matrix is factorized only once (or efficiently re-analyzed), this time tends to be constant.
- For the **Iterative Methods (SOR)**, this time represents the duration of the `while` loop until convergence is reached (residual $< 10^{-5}$). This value is sensitive to the software implementation (Script vs C++/MEX).

The performance comparison (Table 2) highlights how the compiled implementation makes the iterative method competitive.

Solver Type	**Average Time per Step [s]**
Direct	0.02949
SOR	0.08327
SOR (C++ MEX)	0.00954

Table 2: Indicative comparison of calculation times for the Poisson resolution.

6.3 Comparative Analysis of Time Integration Strategies

In this section, the impact of the time integration scheme on the stability and quality of the physical solution is evaluated, while keeping the spatial solver fixed. Three schemes were compared, whose update laws (denoting the spatial residual as $\mathcal{F}(\boldsymbol{\zeta})$) are defined as follows:

1. **Runge-Kutta 4 (RK4):** Explicit *single-step* method. The solution at step $n+1$ is obtained as a weighted average of four intermediate stages ($k_1 \dots k_4$):

$$\boldsymbol{\zeta}^{n+1} = \boldsymbol{\zeta}^n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4) \tag{43}$$

It is a robust but expensive method, requiring **4 functional evaluations** per step.

2. **Adams-Bashforth 2 (AB2):** Second-order linear *multi-step* method. It utilizes the "history" of the solution at steps n and $n-1$:

$$\boldsymbol{\zeta}^{n+1} = \boldsymbol{\zeta}^n + \frac{\Delta t}{2}\left(3\mathcal{F}(\boldsymbol{\zeta}^n) - \mathcal{F}(\boldsymbol{\zeta}^{n-1})\right) \tag{44}$$

.

3. **Adams-Bashforth 3 (AB3):** Third-order *multi-step* method, which extends the history up to $n-2$ to increase formal accuracy:

$$\boldsymbol{\zeta}^{n+1} = \boldsymbol{\zeta}^n + \frac{\Delta t}{12}\left(23\mathcal{F}(\boldsymbol{\zeta}^n) - 16\mathcal{F}(\boldsymbol{\zeta}^{n-1}) + 5\mathcal{F}(\boldsymbol{\zeta}^{n-2})\right) \tag{45}$$

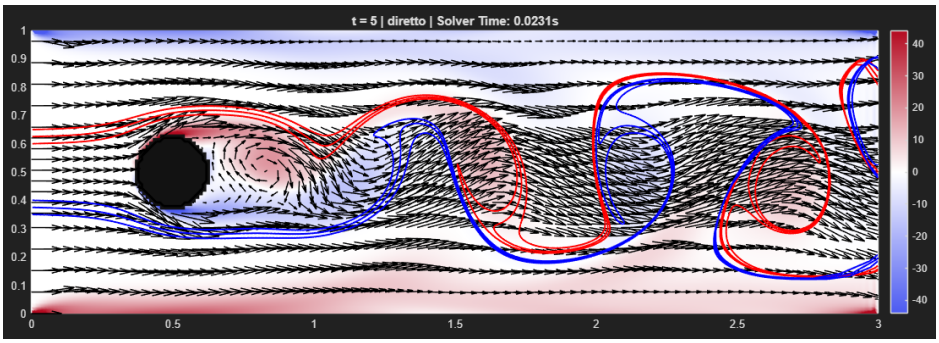
6.3.1 Stability and Robustness Comparison

Simulations conducted with reference parameters ($Re = 500$ and $Cou = 0.2$) showed that all three methods converge to a physically coherent solution. In this conservative regime, the *streaklines* produced by AB2 and AB3 are nearly indistinguishable from those of RK4 (Figure 5), confirming that if the time step is sufficiently small, the truncation error of multi-step methods is acceptable.
However, differences emerge drastically when analyzing stability margins. A sensitivity analysis was performed by increasing the Courant number from 0.3 to 0.4 (corresponding to an increase in the time step Δt, making the stability condition more stringent).

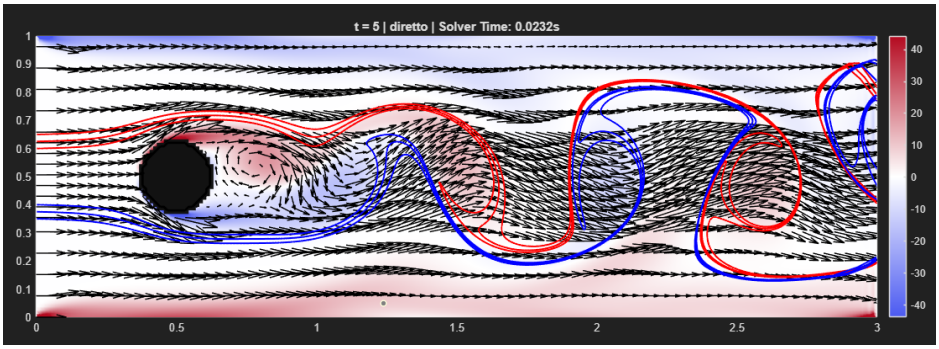
- **Instability of AB methods:** With $Cou = 0.3$, the Adams-Bashforth 3 (AB3) scheme fails. The solution diverges rapidly after a few time instants, driving global enstrophy to infinite values. This behavior is consistent with theory: explicit multi-step methods possess reduced stability regions, making them vulnerable in convection-dominated problems. With $Cou = 0.4$, however, both methods (AB2, AB3) fail.

- **Robustness of RK4:** Under the same conditions ($Cou = 0.4$), the RK4 scheme remains stable. Although an increase in the initial vorticity peak is observed (due to the lower temporal resolution of the startup singularity), the simulation overcomes the transient and settles correctly.

This analysis confirms that, despite the computational cost per single step of RK4 being four times that of AB schemes, it allows for the use of larger time steps without risk of divergence, resulting overall in the most reliable choice for this type of flow.



(a) RK4 Solution



(b) Adams-Bashforth 3 Solution

Figure 5: Visualization of streaklines in the stable regime ($Cou = 0.2$). Vertical comparison between the RK4 solution (a) and that obtained with AB3 (b).

6.4 Overall Comparison and Conclusions

From the comparative analysis, a clear picture emerges regarding the optimal solution strategies for this type of problem:

1. **Poisson Solver:** The use of a SOR iterative method implemented in a compiled language (C++/MEX) represents the most scalable solution,

combining low memory usage with computation times comparable to direct methods.

2. **Time Integration: RK4** confirms itself as the most robust choice. Although computationally more expensive per single step compared to Adams-Bashforth methods, its wide stability region allows for longer time steps and guarantees a physical solution free of spurious oscillations.

7 Presentation of the Code

```
% Codice per la simulazione delle equazioni di Navier-Stokes 2D nella
% formulazione vorticita'-funzione di corrente (Psi-Zita).
% Le equazioni sono integrate in una regione non connessa interna ad un
% rettangolo con un 'ostacolo' nel dominio.
%
%        ----------------------------------------------------
%        ->                                                  ->
%        ->                                                  ->
%        ->              -----                               ->
%        ->            -       -                             ->
%        ->            -       -                             ->
%        ->              -----                               ->
%        ->                                                  ->
%        ->                                                  ->
%        ----------------------------------------------------
%
%
% La collocazione delle variabili sul mesh e' di tipo 'collocated', ovvero
% tutte le variabili sono collocate nei nodi di griglia, che cadono anche
% direttamente sul bordo.
%
% La indicizzazione delle variabili segue una notazione standard nella
% quale la variabile x e' associata all'indice i, mentre la variabile y e'
% associata all'indice j
%
%
%                                  Psi(i,j+1)
%                                      o
%                                      |
%                                      |
%                                      |
%                  Psi(i-1,j) o-------o-------o Psi(i+1,j)
```

```
%                                    Psi(i,j)
%                                       |
%                                       |
%                                       o
%                                    Psi(i,j-1)
close all; clear; clc;

global V U h hq Re G Nx Ny conv om tol
% ---------------------------

method = "diretto";    % Scegli il metodo per la discretizzazione spaziale: "diretto" (delsq) oppure "iterativo_sor/iterativo_sor_cpp" (SOR)
conv   = 'skew';       % Discretizzazione spaziale Schema convettivo
time_method = "AB3";   % metodo temporale: scegli: "RK4", "AB2", "AB3"
om     = 1.8;          % Parametro di rilassamento per SOR (1 < om < 2)
tol    = 1e-5;         % Tolleranza per il metodo iterativo
% ---------------------------

% Domain geometry
Ly = 1;
AR = 3;
Lx = AR * Ly;
Nhy = 79;
Nhx = AR * Nhy;
Ny = Nhy + 1;
Nx = Nhx + 1;
y = linspace(0, Ly, Ny);
x = linspace(0, Lx, Nx);
h = x(2) - x(1);
hq = h*h;
[X, Y] = meshgrid(x, y);

% Boundary conditions su psi
Q = 1;
PsiN = Q * ones(1, Nx)';
PsiS = zeros(1, Nx)';
```

```
68  PsiW = linspace(0, Q, Ny)';
69  PsiE = PsiW;
70  PsiCyl = Q / 2;
71
72  % Physical parameters
73  T = 5;
74  Re = 500;
75  Cou = 0.2;
76  beta = 0.49;
77  uRef = Q / Ly;
78  Dt = min(Cou * h / uRef, beta * Re * hq);
79  Nt = ceil(T / Dt) + 1;
80  t = linspace(0, T, Nt);
81  Dtt = t(2) - t(1);
82
83  % Obstacle geometry (Cylinder)
84  Center = [Lx/6, Ly/2];
85  Radius = Ly / 8;
86
87  % Domain topology and discrete Laplacian operator
88  k = 0;
89  G = zeros(Nx, Ny);
90  for j = 2 : Ny-1
91      for i = 2 : Nx-1
92          if norm([x(i), y(j)] - Center) <= Radius  %=if (x(i)-center
        (1))^2 + (y(j) - center(2))^2 < radius^2
93              G(i, j) = 0;
94          else
95              k = k + 1;
96              G(i, j) = k;
97          end
98      end
99  end
100
101 % PREPARAZIONE SOLUTORE
102 Lap = [];
103 if method == "diretto"
104     disp('Inizializzazione metodo DIRETTO (Matrice Laplaciana)...')
        ;
105     Lap = -delsq(G) / hq;
```

```
106 else
107     disp('Inizializzazione metodo ITERATIVO (SOR)...');
108 end
109
110 % Set G=-1 inside obstacle for identification
111 for i = 2 : Nx-1
112     for j = 2 : Ny-1
113         if G(i, j) == 0
114             G(i, j) = -1;
115         end
116     end
117 end
118
119 % Domain and obstacle visualization
120 figurePosition = [0, 50, 1640, 1640/AR];
121 f = figure('Position', figurePosition, 'Units', 'pixels');
122 D = zeros(Nx, Ny);
123 D(G == -1) = 1;
124 D(G == 0) = 1;
125 D = D';
126 spy(D);
127 xlabel('\it i'); ylabel('\it j');
128 set(gca, 'YDir', 'normal', 'FontSize', 18);
129 drawnow;
130
131 % Array initialization
132 Ninc = k;  %numero di incognite
133 zita = zeros(Ninc, 1);
134 PSI = zeros(Nx, Ny);
135 ZITA = PSI;
136 U = PSI;
137 V = PSI;
138
139 % Boundary conditions into the arrays
140 U(1, :) = uRef;         U(end, :) = uRef;    %velocita' costante
    parete O-E
141 PSI(1, :) = PsiW;       PSI(end, :) = PsiE;
142 PSI(:, 1) = PsiS;       PSI(:, end) = PsiN;  %parete SUD psi=0,
    NORD psi=Q
143 PSI(G == -1) = PsiCyl;  %=Q/2
```

```
144
145 % Pathlines initial condition
146 xP{1} = [0; Ly/2 + 0.100]; xP{2} = [0; Ly/2 + 0.125]; xP{3} = [0;
    Ly/2 + 0.150];
147 xP{4} = [0; Ly/2 - 0.100]; xP{5} = [0; Ly/2 - 0.125]; xP{6} = [0;
    Ly/2 - 0.150];
148
149 % 6 Streaklines initialization
150 for s=1:6
151     xS{s} = zeros(2, Nt);
152     xS{s}(:, 1) = xP{s};
153     xS{s}(:, :) = repmat(xP{s}, 1, Nt);
154 end
155
156 % Graphics preparation
157 clf(f);
158 Uquiv = zeros(Nx, Ny); Vquiv = zeros(Nx, Ny);
159 iq = 1 : 3 : Nx;
160 jq = [1 : 6 : floor(Ny/3), floor(Ny/3+3) : 3 : floor(2*Ny/3), floor
    (2*Ny/3+6) : 6 : Ny];
161 Map = [linspace(75/255, 1, 500)', linspace(90/255, 1, 500)',
    linspace(241/255, 1, 500)'; ...
162        linspace(1, 180/255, 500)', linspace(1, 10/255, 500)',
    linspace(1, 32/255, 500)'];
163
164 % Variabili per analisi
165 enstrophy = NaN(Nt, 1);        %per valutare la conservativita'
    dell'estrofia
166 time_per_step = zeros(Nt, 1);  %per valutare la velocita' del
    metodo
167
168
169 %%modifica per AB
170 RHS_hist = cell(3,1);
171 % RHS_hist{1} = f^{n-2}
172 % RHS_hist{2} = f^{n-1}
173 % RHS_hist{3} = f^{n}
174
175
176 % Time integration
```

```
177 disp('Avvio integrazione temporale...');
178 for it = 1 : Nt
179     time = t(it);
180 
181     ZITA(G == -1) = 0;
182 
183     % Vorticita' al bordo (Thom)
184     ZITA = ThomFormulae(ZITA, PSI, G, hq);    %cc sulla zita
185 
186 
187     % Integrazione Eq. Vorticita' (RK4  - AB2 - AB3)
188 
189     % === Calcolo RHS corrente (f^n) ===
190     RHSn = ZitaRHS(time, ZITA, Re, h, hq, U, V, G, Nx, Ny);
191 
192     switch time_method
193 
194         case "RK4"
195             ZITA = RK4(time, ZITA, @(t,y) ZitaRHS(t,y,Re,h,hq,U,V,G
    ,Nx,Ny), Dt);
196 
197         case "AB2"
198             if it <= 2
199                 % Startup con RK4
200                 ZITA = RK4(time, ZITA, @(t,y) ZitaRHS(t,y,Re,h,hq,U
    ,V,G,Nx,Ny), Dt);
201 
202                 % Riempio storico progressivamente
203                 RHS_hist{it} = RHSn;
204             else
205                 % AB2: usa f^n e f^{n-1}
206                 ZITA = AB2step(ZITA, RHSn, RHS_hist{2}, Dt);
207 
208                 % Shift storico
209                 RHS_hist{1} = RHS_hist{2};
210                 RHS_hist{2} = RHSn;
211             end
212 
213         case "AB3"
214             if it <= 3
```

```
215                 % Startup con RK4
216                 ZITA = RK4(time, ZITA, @(t,y) ZitaRHS(t,y,Re,h,hq,U
    ,V,G,Nx,Ny), Dt);
217
218                 RHS_hist{it} = RHSn;
219             else
220                 % AB3: usa f^n, f^{n-1}, f^{n-2}
221                 ZITA = AB3step(ZITA, RHSn, RHS_hist{3}, RHS_hist
    {2}, Dt);
222
223                 % Shift storico
224                 RHS_hist{1} = RHS_hist{2};
225                 RHS_hist{2} = RHS_hist{3};
226                 RHS_hist{3} = RHSn;
227             end
228
229         otherwise
230             error("Metodo temporale non riconosciuto");
231     end
232
233
234     % Risoluzione Ellittica per PSI (Poisson)
235     tic; % Start timer risolutore
236
237     if method == "diretto"
238         % Metodo Diretto (si usa delsq classico)
239         [PSI, zita] = zita2psi(zita, ZITA, PSI, Lap, G, Nx, Ny, hq,
     PsiS, PsiN, PsiW, PsiE, PsiCyl);
240     elseif method == "iterativo_sor"
241         % Metodo Iterativo (SOR)
242         % Nota: Non restituisce 'zita' vettore, ma aggiorna
    direttamente PSI matrice
243         PSI = PoissonSolver_SOR(ZITA, PSI, G, PsiS, PsiN, PsiW,
    PsiE, PsiCyl);
244     else
245         PSI = PoissonSolver_SOR_CPP(ZITA, PSI, G, PsiS, PsiN, PsiW,
     PsiE, PsiCyl, hq, om, tol);
246     end
247
248     time_per_step(it) = toc; % Stop timer
```

```
249
250     % Calcolo Enstrofia Globale (uguale per entrambi i casi)
251     % Integrale di zita^2 sul dominio fluido
252     enstrophy(it) = 0.5 * sum(ZITA(G > 0).^2) * hq;
253
254     % Calcolo Velocit
255     [U, V] = psi2uv(PSI, U, V, G, h, Nx, Ny, uRef);
256
257     %  Aggiornamento Streaklines
258     for k = 1 : it
259         for s=1:6
260             xS{s}(:, k) = RK4(time, xS{s}(:, k), @(t, y_)
    PathlinesRHS(t, y_, x, y, U, V, h, h, Nx, Ny), Dt);
261         end
262     end
263
264
265     % Plotting ogni 10 step
266     if mod(it, 10) == 0
267         ZITAg = ZITA;
268         ZITAg(G == -1) = NaN;
269
270         subplot(2,1,1); % Plot fluido sopra
271         pcolor(x, y, ZITAg');
272         colormap(Map);
273         shading interp;
274
275         vals = ZITA(G > 0); vals = vals(~isnan(vals));
276         if ~isempty(vals)
277             c_min = 0.7 * min(vals); c_max = 0.7 * max(vals);
278             if c_min < c_max; clim([c_min, c_max]); end
279         end
280         colorbar; hold on;
281
282         Uquiv(iq, jq) = U(iq, jq);
283         Vquiv(iq, jq) = V(iq, jq);
284         quiver(X(jq, iq), Y(jq, iq), Uquiv(iq, jq)', Vquiv(iq, jq)
    ', 3, 'Color', 'k');
285
286         % Disegno ostacolo generico
```

```
287         contour(X, Y, G', [0 0], 'k', 'LineWidth', 2);
288
289         % Plot Streaklines
290         indices_plot = linspace(1, it, min(it*10, 2000));
291         for s = 1:6
292             if it > 1
293                 % Filtro NaN per evitare crash interpolazione
294                 Xpts = xS{s}(1, 1:it); Ypts = xS{s}(2, 1:it);
295                 if any(isnan(Xpts)) || any(isnan(Ypts)); continue;
    end
296
297                 x_plot = interp1(1:it, Xpts, indices_plot, 'pchip')
    ;
298                 y_plot = interp1(1:it, Ypts, indices_plot, 'pchip')
    ;
299                 col = 'r'; if s > 3; col = 'b'; end
300                 plot(x_plot, smooth(y_plot), 'LineWidth', 1.5, '
    Color', col);
301             end
302         end
303
304         title(['t = ', num2str(time, 2), ' | ', char(method), ' |
    Solver Time: ', num2str(time_per_step(it), '%.4f'), 's']);
305         axis image; axis([0, Lx, 0, Ly]);
306         hold off;
307
308         subplot(2,1,2); % Plot enstrofia sotto
309         plot(t(1:it), enstrophy(1:it), 'r', 'LineWidth', 2.5);
310         title('Evoluzione Enstrofia Globale');
311         xlabel('t'); ylabel('\Omega'); grid on; xlim([0, T]);
312
313         drawnow limitrate;
314     end
315 end
316
317 avg_time = mean(time_per_step);
318 fprintf('Simulazione conclusa. Metodo: %s. Tempo medio solver: %.5f
     s\n', method, avg_time);
```

```
function yNew = RK4(t, yOld, RHSfunction, Dt)
    t1 = t;
    y1 = yOld;
    f1 = RHSfunction(t1, y1);

    t2 = t + Dt / 2;
    y2 = yOld + Dt * 0.5 * f1;
    f2 = RHSfunction(t2, y2);

    t3 = t + Dt / 2;
    y3 = yOld + Dt * 0.5 * f2;
    f3 = RHSfunction(t3, y3);

    t4 = t + Dt;
    y4 = yOld + Dt * f3;
    f4 = RHSfunction(t4, y4);

    yNew = yOld + Dt * ((f1 + f4)./6 + (f2 + f3)./3);
end
```

```
function dxdt = PathlinesRHS(t, xP, x, y, U, V, Dx, Dy, Nx, Ny)

    %localizzazione deglla cella in cui si trova la particella
    i = floor(xP(1) / Dx) + 1;
    j = floor(xP(2) / Dy) + 1;

    % limiti indici
    i = max(1, min(i, Nx-1));
    j = max(1, min(j, Ny-1));

    %coordinate del nodo in basso a sinistra deella cella in cui si
     trova la particella
    xNode = [x(i); y(j)];

    %adimensionalizzazione della posizione della particella q nella
     cella
    csix = (xP(1) - xNode(1)) / Dx;
    csiy = (xP(2) - xNode(2)) / Dy;

    if i+1 <= Nx && j+1 <= Ny && i > 0 && j > 0
        % Interpolazione bilineare
        u = [1-csix, csix] * U([i, i+1], [j, j+1]) * [1-csiy; csiy
    ];
        v = [1-csix, csix] * V([i, i+1], [j, j+1]) * [1-csiy; csiy
    ];
    else
        u = 0.05;
        v = 0;
    end

    dxdt = [u; v];
end
```

```
function F = ZitaRHS(t, ZITA, Re, h, hq, U, V, G, Nx, Ny)
% RHS della equazione per Zita per il codice TunnelPsiZita
global conv
F = zeros(Nx,Ny);
for i = 2:Nx-1
    for j = 2:Ny-1
% Convective term
        if G(i,j)>0
            switch conv
                case 'div'
                    F(i,j) = -((U(i+1,j)*ZITA(i+1,j)-U(i-1,j)*ZITA(
    i-1,j))/(2*h)+...
                                (V(i,j+1)*ZITA(i,j+1)-V(i,j-1)*ZITA(
    i,j-1))/(2*h) );
                case 'adv'
                    F(i,j) = -( U(i,j)*(ZITA(i+1,j)-ZITA(i-1,j))
    /(2*h)+...
                                V(i,j)*(ZITA(i,j+1)-ZITA(i,j-1))
    /(2*h) );
                case 'skew'
                    F(i,j) = -0.5*((U(i+1,j)*ZITA(i+1,j)-U(i-1,j)*
    ZITA(i-1,j))/(2*h)+...
                                   (V(i,j+1)*ZITA(i,j+1)-V(i,j-1)*
    ZITA(i,j-1))/(2*h) );
                    F(i,j) = F(i,j)-0.5*(U(i,j)*(ZITA(i+1,j)-ZITA(i
    -1,j))/(2*h)+...
                                         V(i,j)*(ZITA(i,j+1)-ZITA(i
    ,j-1))/(2*h) );
            end
% Diffusive term
            F(i,j) = F(i,j) + (1/Re)*(ZITA(i+1,j)+ZITA(i-1,j)+ZITA(
    i,j+1)+...
                                      ZITA(i,j-1) - 4*ZITA(i,j))/hq
    ;
        end
    end
end
end
```

```
1
2  function [PSI, zita] = zita2psi(zita, ZITA, PSI, Lap, G, Nx, Ny, hq
       , PsiS, PsiN, PsiW, PsiE, PsiCyl)
3
4      % Inizializzazione: Mappatura della vorticita' (ZITA 2D) nel
       vettore termine noto (zita 1D)
5      for i = 2:Nx-1
6          for j = 2:Ny-1
7              if G(i,j) > 0
8                  % Mettiamo il valore di vorticita' nel punto k
       corrispondente
9                  zita(G(i,j)) = ZITA(i,j);
10             end
11         end
12     end
13
14     % Modifica del termine noto per le Condizioni al Contorno (BCs)
15     % Ciclo su tutti i nodi interni per controllare i vicini
16     for i = 2:Nx-1
17         for j = 2:Ny-1
18             if G(i,j) > 0       % Se siamo in un nodo fluido
19                 k = G(i,j);     % Indice lineare nel sistema
       risolutivo
20
21                 % --- Controllo Pareti Esterne (G == 0) ---
22
23                 % Parete Sud (vicino j-1 e' bordo cioe' e' 0)
24                 if G(i,j-1) == 0; zita(k) = zita(k) - PsiS(i)/hq;
       end
25                 % Parete Nord (vicino j+1 e' bordo 0)
26                 if G(i,j+1) == 0; zita(k) = zita(k) - PsiN(i)/hq;
       end
27                 % Parete Ovest (vicino i-1 e' bordo 0)
28                 if G(i-1,j) == 0;  zita(k) = zita(k) - PsiW(j)/hq;
       end
29                 % Parete Est (vicino i+1 e' bordo 0)
30                 if G(i+1,j) == 0;  zita(k) = zita(k) - PsiE(j)/hq;
       end
31
```

```
32                  % Controllo Pareti Ostacolo (G == -1)
33                  % Se un vicino e' ostacolo, sottraiamo il valore
    costante PsiCyl
34
35                  if G(i,j+1) == -1;  zita(k) = zita(k) - PsiCyl/hq;
        end
36                  if G(i,j-1) == -1;  zita(k) = zita(k) - PsiCyl/hq;
        end
37                  if G(i+1,j) == -1;  zita(k) = zita(k) - PsiCyl/hq;
        end
38                  if G(i-1,j) == -1;  zita(k) = zita(k) - PsiCyl/hq;
        end
39              end
40          end
41      end
42
43      % Risoluzione dell'equazione di Poisson (Sistema Lineare) con
    metodo diretto
44      % Lap e' la matrice del Laplaciano discreto, zita e' il termine
     noto modificato
45      psi = Lap \ zita;
46
47      % Riversamento del risultato nel vettore 2D PSI
48      PSI(G > 0) = psi;
49  end
```

```
1
2  function PHI = PoissonSolver_SOR(Zita, PSI_old, G, PsiS, PsiN, PsiW
       , PsiE, PsiCyl)
3  % Risolutore iterativo SOR per l'equazione di Poisson nabla^2 psi =
       zita
4  % Le BCs sono Dirichlet non omogenee (PHI = cost != 0).
5
6  global Nx Ny hq om tol
7
8  PHI = PSI_old;
9  iter = 0;
10 normres = 1;
11 max_iter = 2000; % Limite sicurezza per evitare loop infiniti
12
13 while normres > tol && iter < max_iter
14     iter = iter + 1;
15     max_res = 0; % Per calcolare la norma infinito del residuo
16
17     for i = 2:Nx-1
18         for j = 2:Ny-1
19             if G(i,j) > 0 % Calcola solo sui nodi fluidi
20
21                 %  Si recuperano i valori dai vicini (Gestione BCs)
       , in particolare se il vicino e' un punto del bordo allora si
       prende il valore noto di PSI altrimenti
22                 % si prende il valore in quel puunti di PSI.
23
24                 % EST (i+1)
25                 if G(i+1,j) > 0;      valE = PHI(i+1,j);   % Fluido
26                 elseif G(i+1,j) == 0; valE = PsiE(j);      % Bordo
       Est
27                 else;                 valE = PsiCyl;       %
       Ostacolo
28                 end
29
30                 % OVEST (i-1)
31                 if G(i-1,j) > 0;      valW = PHI(i-1,j);   % Fluido
32                 elseif G(i-1,j) == 0; valW = PsiW(j);      % Bordo
       Ovest
```

```
33              else;                   valW = PsiCyl;       % Ostacolo
34              end
35
36              % NORD (j+1)
37              if G(i,j+1) > 0;      valN = PHI(i,j+1);   % Fluido
38              elseif G(i,j+1) == 0; valN = PsiN(i);      % Bordo Nord
39              else;                   valN = PsiCyl;       % Ostacolo
40              end
41
42              % SUD (j-1)
43              if G(i,j-1) > 0;      valS = PHI(i,j-1);   % Fluido
44              elseif G(i,j-1) == 0; valS = PsiS(i);      % Bordo Sud
45              else;                   valS = PsiCyl;       % Ostacolo
46              end
47
48              % Formula di Aggiornamento SOR
49              % Laplaciano discreto: (E + W + N + S - 4P)/h^2 = Zita
50              % P_new = (1-om)*P_old + om * 0.25 * (E + W + N + S - h^2*Zita)
51
52              RHS_Term = hq * Zita(i,j);
53              Phi_Star = 0.25 * (valE + valW + valN + valS - RHS_Term);
54
55              % Calcolo residuo locale (solo ogni 10 iterazioni per velocita')
56              if mod(iter, 10) == 0
57                  res_loc = abs(Phi_Star - PHI(i,j));
58                  if res_loc > max_res; max_res = res_loc; end
59              end
60
61              % Aggiornamento
62              PHI(i,j) = (1 - om) * PHI(i,j) + om * Phi_Star;
63
```

```
64              end
65          end
66      end
67
68          if mod(iter, 10) == 0
69              normres = max_res;
70          end
71      end
72  end
```

```
function [U, V] = psi2uv(PSI, U, V, G, h, Nx, Ny, uRef)
    i = 2:Nx-1;   j = 2:Ny-1;

    U(i, j) = (PSI(i, j+1) - PSI(i, j-1)) / (2 * h);
    V(i, j) = -(PSI(i+1, j) - PSI(i-1, j)) / (2 * h);

    U(G < 0) = 0;
    V(G < 0) = 0;

    U(1, :) = uRef;
    U(end, :) = uRef;
end
```

```
function yNew = AB2step(yN, fN, fNm1, Dt)
% Adams-Bashforth 2-step explicit
% y^{n+1} = y^n + Dt*(3/2 f^n - 1/2 f^{n-1})

yNew = yN + Dt*(1.5*fN - 0.5*fNm1);

end
```

```
function yNew = AB3step(yN, fN, fNm1, fNm2, Dt)
% Adams-Bashforth 3-step explicit
% y^{n+1} = y^n + Dt*(23/12 f^n - 16/12 f^{n-1} + 5/12 f^{n-2})

yNew = yN + Dt*( (23/12)*fN - (16/12)*fNm1 + (5/12)*fNm2 );

end
```

References

[1] J. C. Tannehill, D. A. Anderson and R. H. Pletcher, *Computational Fluid Mechanics and Heat Transfer*, 2nd edition. CRC Press (1997).

List of Figures

List of Tables